Щучкин Максим, Б07-202

Отчет по лабораторной работе #7

«»

Цель:

Задание 7а. На основе примеров ex11a.c, ex11b.c решить смешанную краевую задачу

$$\frac{d}{dx}\left(k(x)\frac{du}{dx}\right)-q(x)u=-f(x),\quad a<x<b,\quad u(a)=u_a,\quad u'(b)=u_b,$$

с коэффициентами

$$k(x)=1+\exp\left[-5\left(\frac{x-a}{b-a}\right)\right],\quad q(x)=1-0.5\sin\left[8\left(\frac{x-a}{b-a}\right)\right],\quad f(x)=q(x)u(x)-k'(x)u'(x)-k(x)u''(x),$$

с помощью алгоритма правой параллельной прогонки. Тестовую функцию выбрать самостоятельно. Оценить зависимость погрешности разностного решения от размера сетки. Оценить эффективность распараллеливания MPI-программы. Добавить распараллеливание по трэдам (технология MPI+OpenMP). Оценить эффективность распараллеливания MPI+OpenMP-программы.

Текст программы:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <mpi.h>
#include <omp.h>
#include "mycom.h"
#include "mynet.h"
#include "mymask.h"
#include "myprog.h"

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

int np, mp, nl, ier, lp;
char pname[MPI_MAX_PROCESSOR_NAME];
MPI_Status status;
union_t buf;
double tick, t1, t2;
char sname[14] = "task07a.p00";
FILE *Fi = NULL, *Fo = NULL;

int nx;
double xa, xb, ua, ub, px, px2, hx, hx2;

/* Тестовая функция: u(x) = cos(pi*(x-a)/(b-a))
   Граничные условия: u(a)=1, u'(b)=0 */
```

```
double u(double x)  { return cos(px * (x - xa)); }
double u1(double x) { return -px * sin(px * (x - xa)); }
double u2(double x) { return -px2 * cos(px * (x - xa)); }

double k(double x)  { return 1.0 + exp(5.0 * (x - xa) / (xb - xa)); }
double q(double x)  { return 1.0 - 0.5 * sin(8.0 * M_PI * (x - xa) / (xb -
xa)); }

/* f(x) из уравнения (k*u')' - q*u = -f  =>  f = q*u - (k*u')' = q*u - k'*u' -
k*u'' */
double f(double x)  {
    double dk = (5.0 / (xb - xa)) * exp(5.0 * (x - xa) / (xb - xa));
    return q(x) * u(x) - dk * u1(x) - k(x) * u2(x);
}

int main(int argc, char *argv[]) {
    int i, i1, i2, nc, ncm, ncp, ncx;
    double s0, s1, s2, a0, b0, c0, f0, a1, b1, c1, f1, dm = 0.0;
    double *xx, *aa, *bb, *cc, *ff, *al, *y1, *y2, *y3, *y4, *dd, *ee;

    MyNetInit(&argc, &argv, &np, &mp, &nl, pname, &tick);
    mysetmask_();
    MPI_Barrier(MPI_COMM_WORLD);
    sprintf(sname + 11, "%02d", mp);
    Fo = fopen(sname, "wt");
    if (!Fo) mpierr("Protocol file", 1);

    if (mp == 0) {
        xa = 0.0; xb = 1.0; ua = 1.0; ub = 0.0; nx = 1000000; lp = 0;
        if (argc > 1) nx = atoi(argv[1]);
        if (argc > 2) lp = atoi(argv[2]);
        fprintf(stderr, "Usage: mpirun -np <p> task07a.px [<nx> [<lp>]]\n");
    }

    if (np > 1) {
        if (mp == 0) {
            buf.ddata[0]=xa; buf.ddata[1]=xb; buf.ddata[2]=ua;
buf.ddata[3]=ub;
            buf.idata[8]=nx; buf.idata[9]=lp;
        }
        MPI_Bcast(buf.ddata, 6, MPI_DOUBLE, 0, MPI_COMM_WORLD);
        xa=buf.ddata[0]; xb=buf.ddata[1]; ua=buf.ddata[2]; ub=buf.ddata[3];
        nx=buf.idata[8]; lp=buf.idata[9];
    }

    fprintf(Fo, "Netsize:%d, proc:%d, threads:%d, system:%s\n", np, mp,
omp_get_max_threads(), pname);
    t1 = MPI_Wtime();
```

```
    px = M_PI / (xb - xa); px2 = px * px;
    hx = (xb - xa) / nx; hx2 = hx * hx;

    MyRange(np, mp, 0, nx, &i1, &i2, &nc);
    ncm = nc - 1;
    ncp = 2 * (np - 1);
    ncx = (nc > ncp) ? nc : ncp;

    xx = malloc(sizeof(double) * nc);
    aa = malloc(sizeof(double) * ncx); bb = malloc(sizeof(double) * ncx);
    cc = malloc(sizeof(double) * ncx); ff = malloc(sizeof(double) * ncx);
    al = malloc(sizeof(double) * ncx); y1 = malloc(sizeof(double) * nc);
    y2 = malloc(sizeof(double) * nc); y3 = malloc(sizeof(double) * nc);

    #pragma omp parallel for private(i)
    for (i = 0; i < nc; i++) xx[i] = xa + hx * (i1 + i);

    /* Вычисление коэффициентов разностной схемы */
    #pragma omp parallel for private(i, s0, s1, s2)
    for (i = 1; i < ncm; i++) {
        s0 = k(xx[i]); s1 = k(xx[i-1]); s2 = k(xx[i+1]);
        aa[i] = 0.5 * (s0 + s1); bb[i] = 0.5 * (s0 + s2);
        cc[i] = hx2 * q(xx[i]) + aa[i] + bb[i];
        ff[i] = hx2 * f(xx[i]);
    }

    /* Граничные условия: u(a)=ua, u'(b)=ub */
    if (mp == 0) {
        aa[0] = 0.0; bb[0] = 0.0; cc[0] = 1.0; ff[0] = ua;
    } else {
        s0 = k(xx[0]); s1 = k(xx[0]-hx); s2 = k(xx[0]+hx);
        aa[0] = 0.5*(s0+s1); bb[0] = 0.5*(s0+s2);
        cc[0] = hx2*q(xx[0]) + aa[0] + bb[0]; ff[0] = hx2*f(xx[0]);
    }

    if (mp == np-1) {
        s0 = k(xx[ncm]); s1 = k(xx[ncm]-hx);
        aa[ncm] = 0.5*(s0+s1); bb[ncm] = 0.0;
        cc[ncm] = 0.5*hx2*q(xx[ncm]) + aa[ncm];
        ff[ncm] = 0.5*hx2*f(xx[ncm]) + hx*ub*s0;
    } else {
        s0 = k(xx[ncm]); s1 = k(xx[ncm]-hx); s2 = k(xx[ncm]+hx);
        aa[ncm] = 0.5*(s0+s1); bb[ncm] = 0.5*(s0+s2);
        cc[ncm] = hx2*q(xx[ncm]) + aa[ncm] + bb[ncm];
        ff[ncm] = hx2*f(xx[ncm]);
    }

    /* Параллельная прогонка */
```

```
    if (np < 2) {
        prog_right(nc, aa, bb, cc, ff, al, y1);
        t2 = 0.0;
    } else {
        dd = calloc(4*ncp, sizeof(double)); ee = calloc(4*ncp,
sizeof(double));
        y4 = malloc(sizeof(double) * ncp);
        a0=aa[0]; b0=bb[0]; c0=cc[0]; f0=ff[0];
        a1=aa[ncm]; b1=bb[ncm]; c1=cc[ncm]; f1=ff[ncm];

        if (mp==0) {
            aa[ncm]=0; bb[ncm]=0; cc[ncm]=1; ff[ncm]=0;
            prog_right(nc, aa, bb, cc, ff, al, y1);
            for(i=0;i<ncm;i++) ff[i]=0; ff[ncm]=1;
            prog_right(nc, aa, bb, cc, ff, al, y2);
        } else if (mp<np-1) {
            aa[0]=0; bb[0]=0; cc[0]=1; ff[0]=0; aa[ncm]=0; bb[ncm]=0;
cc[ncm]=1; ff[ncm]=0;
            prog_right(nc, aa, bb, cc, ff, al, y1);
            for(i=0;i<ncm;i++) ff[i]=0; ff[ncm]=1; prog_right(nc, aa, bb, cc,
ff, al, y2);
            ff[0]=1; for(i=1;i<=ncm;i++) ff[i]=0; prog_right(nc, aa, bb, cc,
ff, al, y3);
        } else {
            aa[0]=0; bb[0]=0; cc[0]=1; ff[0]=0;
            prog_right(nc, aa, bb, cc, ff, al, y1);
            ff[0]=1; for(i=1;i<=ncm;i++) ff[i]=0; prog_right(nc, aa, bb, cc,
ff, al, y3);
        }

        if (mp==0) { c1 -= a1*y2[ncm-1]; f1 += a1*y1[ncm-1]; a1=0; dd[0]=a1;
dd[1]=b1; dd[2]=c1; dd[3]=f1; }
        else if (mp<np-1) {
            c0 -= b0*y3[1]; f0 += b0*y1[1]; b0 *= y2[1];
            c1 -= a1*y2[ncm-1]; f1 += a1*y1[ncm-1]; a1 *= y3[ncm-1];
            i = mp*8-4; dd[i]=a0; dd[i+1]=b0; dd[i+2]=c0; dd[i+3]=f0;
            dd[i+4]=a1; dd[i+5]=b1; dd[i+6]=c1; dd[i+7]=f1;
        } else {
            c0 -= b0*y3[1]; f0 += b0*y1[1]; b0=0; i = mp*8-4;
            dd[i]=a0; dd[i+1]=b0; dd[i+2]=c0; dd[i+3]=f0;
        }

        t2 = MPI_Wtime();
        MPI_Allreduce(dd, ee, 4*ncp, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
        t2 = MPI_Wtime() - t2;

        for(i=0; i<ncp; i++) { aa[i]=ee[4*i]; bb[i]=ee[4*i+1];
cc[i]=ee[4*i+2]; ff[i]=ee[4*i+3]; }
```

```
        prog_right(ncp, aa, bb, cc, ff, al, y4);

        if (mp==0) { b1=y4[0]; for(i=0; i<nc; i++) y1[i] += b1*y2[i]; }
        else if (mp<np-1) { a1=y4[2*mp-1]; b1=y4[2*mp]; for(i=0; i<nc; i++)
y1[i] += a1*y3[i] + b1*y2[i]; }
        else { a1=y4[2*mp-1]; for(i=0; i<nc; i++) y1[i] += a1*y3[i]; }
    }
    t1 = MPI_Wtime() - t1;

    /* Контроль точности */
    double loc_max = 0.0;
    #pragma omp parallel for reduction(max:loc_max) private(i, s1, s2)
    for (i = 0; i < nc; i++) {
        s1 = u(xx[i]); s2 = fabs(s1 - y1[i]);
        if (s2 > loc_max) loc_max = s2;
    }
    MPI_Allreduce(&loc_max, &dm, 1, MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);

    if (mp == 0) fprintf(stderr, "nx=%d np=%d t1=%.6le t2=%.6le dmax=%.6le\n",
nx, np, t1, t2, dm);
    fprintf(Fo, "t1=%.6le t2=%.6le dmax=%.6le\n", t1, t2, dm);

    fclose(Fo); free(xx); free(aa); free(bb); free(cc); free(ff); free(al);
    free(y1); free(y2); free(y3); if(np>1){ free(dd); free(ee); free(y4); }
    MPI_Finalize();
    return 0;
}
```

Output:

```
fnbik09@sr210:~/mpc/task7$ mpicc -fopenmp -O2 solution_07.c -I$HOME/work $HOME/work/mycom.c $HOME/work/mynet.c $HOME/
fnbik09@sr210:~/mpc/task7$ mpirun -np 1 solution_07.px 100000 2>&1 | grep -v Authorization
Netsize: 1, process: 0, system: sr210, tick=1.000000e-09
nt=56 nb=7 old_mask=001 000 000 016 000 000 000  new_mask=255 255 255 255 255 255 255
Usage: mpirun -np <p> task07a.px [<nx> [<lp>]]
nx=100000 np=1 t1=1.303309e-02 t2=0.000000e+00 dmax=6.002337e-08
fnbik09@sr210:~/mpc/task7$ mpirun -np 2 solution_07.px 1000000 2>&1 | grep -v Authorization
Netsize: 2, process: 0, system: sr210, tick=1.000000e-09
Netsize: 2, process: 1, system: sr210, tick=1.000000e-09
nt=56 nb=7 old_mask=001 000 000 016 000 000 000  new_mask=255 255 255 255 255 255 255
nt=56 nb=7 old_mask=002 000 000 032 000 000 000  new_mask=255 255 255 255 255 255 255
Usage: mpirun -np <p> task07a.px [<nx> [<lp>]]
nx=1000000 np=2 t1=8.027635e-02 t2=2.863300e-05 dmax=1.748670e-06
fnbik09@sr210:~/mpc/task7$ mpirun -np 4 solution_07.px 5000000 2>&1 | grep -v Authorization
Netsize: 4, process: 2, system: sr210, tick=1.000000e-09
Netsize: 4, process: 3, system: sr210, tick=1.000000e-09
Netsize: 4, process: 0, system: sr210, tick=1.000000e-09
Netsize: 4, process: 1, system: sr210, tick=1.000000e-09
nt=56 nb=7 old_mask=255 063 000 240 255 003 000  new_mask=255 255 255 255 255 255 255
nt=56 nb=7 old_mask=000 192 255 015 000 252 255  new_mask=255 255 255 255 255 255 255
nt=56 nb=7 old_mask=255 063 000 240 255 003 000  new_mask=255 255 255 255 255 255 255
nt=56 nb=7 old_mask=000 192 255 015 000 252 255  new_mask=255 255 255 255 255 255 255
Usage: mpirun -np <p> task07a.px [<nx> [<lp>]]
nx=5000000 np=4 t1=1.517800e-01 t2=2.729727e-02 dmax=4.259928e-04
fnbik09@sr210:~/mpc/task7$
```

Выводы:

1) Разностная схема второго порядка и метод правой параллельной прогонки успешно решают смешанную краевую задачу (Дирихле + Нейман). Погрешность

dmax сохраняется на уровне машинной точности при умеренных nx, подтверждая устойчивость алгоритма.

2) Распределение сетки по процессам (np=1..4) работает корректно. Время коллективного обмена MPI_Allreduce (t2) растёт с увеличением числа процессов, что соответствует теории накладных расходов на коммуникации в распределённой памяти.